An Efficient Implementation of a TAGE Branch Predictor for Soft Processors on FPGA

Katsunoshin Matsui*, Md Ashraful Islam* and Kenji Kise*
*School of Computing, Tokyo Institute of Technology
Tokyo, Japan
Email: {matsui, ashraful, kise}@arch.cs.titech.ac.jp

***Abstract*—Soft processors are becoming a common component on reconfigurable computing like FPGA. For some accelerators, custom logic functions are implemented as processing elements besides the soft processor. Since the resources in FPGA are fixed and limited, it is desired to implement the soft processor with less logical resources as possible. One of the important parts of the processor is an instruction fetch unit whose performance is dependent on branch prediction. Conventional branch predictors like bimodal or gshare are simple to implement but their prediction accuracy is not good enough. On the other hand, TAGE branch predictor has better prediction accuracy but contains complex logic path for branch prediction, which results in the lower operating frequency.**

In this paper, we propose a branch predictor called pTAGE, which has almost the same prediction accuracy as TAGE and avoids becoming the critical path of the processor. The branch prediction of pTAGE is pipelined, so prediction result is available on each clock cycle. We implement gshare, TAGE, and pTAGE, respectively in Verilog HDL and evaluate their operating frequency and prediction rate based on FPGA implementation. In this result, pTAGE has almost the same prediction rate as TAGE and 1.41 times higher operating frequency than that of TAGE. Also, we evaluate the performance by varying the latency for updating branch prediction, and the evaluation result shows that pTAGE exhibits higher performance in deep pipelined processors than gshare.

***Keywords*-FPGA, branch prediction, TAGE, soft processor**

I. Introduction

The cost-effective soft processors for FPGAs like MicroBlaze [1] and Nios II [2] are widely used in various fields. On the other hand, RISC-V [3] is becoming popular as a RISC based open instruction set architecture (ISA) which has been developed at the University of California, Berkeley, and the demand for cost-effective soft processors based on RISC-V will become higher.

To satisfy the demand, we are running the project of developing a cost-effective scalar RISC-V processor. As the first step of this project, we design an instruction fetch unit with a modern branch predictor for conditional branch instructions, where the predictor with a complex structure is hard to be implemented while satisfying the timing constraints of a soft processor. In general, a branch prediction should be completed in a clock cycle, therefore the operating frequency of the branch predictor will be lower, which will decrease the performance of the overall processor.

Although gshare [4] branch predictor introduced in 1993 whose structure is relatively simple is used in some soft processors, the prediction accuracy of gshare is not high compared to some modern and high-performance branch predictors. TAGE (TAgged GEometric length predictor) [5] is one of the high-performance predictors proposed by A. Seznec et al. in 2006. Because of the high accuracy and the suitable structure for FPGA of TAGE predictor, we select to use TAGE predictor for our project.

The other motivation of this work is to clarify the relationship between the update latency and the accuracy of branch predictors where the update latency is defined as the number of cycles between a prediction time and its update time. The sign L denotes the update latency in this work. In a typical five-stage pipelining processor having IF (instruction fetch), ID (instruction decode), EX (execution), MEM (memory access), and WB (write back) stages, a branch prediction is done in the first stage of IF and a predictor update is done in the third stage of EX where the update latency is 2 cycles or L=2. Because the update latency may differ when using different pipeline structure, we evaluate the prediction accuracy and instruction fetch performance varying the update latency.

In this paper, for soft processors on FPGAs, we propose a new implementation of TAGE branch predictor which has almost the same prediction accuracy as TAGE and achieves higher operating frequency. The implementation is pipelined, so a prediction result is available on each clock cycle. The main contributions of this paper are as follows.

- We clarify the problems of implementing a TAGE branch predictor on FPGAs. We show that the operating frequency of a branch predictor like TAGE is low since the branch predictor completes a branch prediction in a clock cycle. We also clarify the influence of the branch predictor updating latency on the accuracy of the branch prediction.
- To solve critical path problems of implementing a TAGE branch predictor, we propose an effective implementation of TAGE branch predictor named *pTAGE*. The proposal runs faster than a conventional TAGE because it makes a branch prediction in two cycles. While keeping the high operating frequency, it can

978-1-7281-4882-3/19/$31.00 ©2019 IEEE
DOI 10.1109/MCSoC.2019.00023

IEEE computer society
Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

make a prediction on every clock cycle because it adopts the pipelining.

- We implement three branch predictors, gshare, TAGE, and pTAGE in Verilog HDL. We also prepare an evaluation environment to check the influence of update latency on branch predictor's accuracy. We evaluate their performances, hardware resources, and operating frequency. Through the evaluation, we show that pTAGE using less embedded memories (BRAMs) than gshare achieves high prediction accuracy and high operating frequency.

II. Background

This section describes the background of this work. Firstly, we show the architecture of an instruction fetch unit we assumed. It is a typical fetch unit having a branch predictor, branch target buffers, and an instruction cache. Secondly, we show the branch prediction model for an FPGA implementation. Then, we describe the organization of a baseline TAGE predictor and challenges for the TAGE implementation.

A. Instruction fetch unit

Figure 1 shows the block diagram of an instruction fetch unit for RISC-V soft processor supporting the compressed instructions. This is a typical fetch unit and we used this unit to evaluate some branch predictors.

This unit has an instruction cache, a branch predictor, and branch target buffers (BTBs) [6]. The memory modules are shown as orange rectangles in the figure. This unit has two BTBs for storing branch target addresses: BTB1 is for conditional branches and BTB2 is for unconditional branches. It also contains combinational units: two adders, a multiplexer named Mux, the module Select which generates the control signal for Mux, and five combinational circuits named C1, C2, C3, C4, and C5. Note that the prefix C indicates a combinational circuit. The registers PC and IR (instruction register) which are shown as red rectangles are triggered by the positive edge clock. The registers PC_a, PC_b, PC_c, and PC_d having the same value as PC are used for the input signal of each module.

To fetch an instruction specified by PC4, the instruction cache is accessed. The cache hit or miss is flagged by signal IC_HIT determined by C5. If cache hits, then a valid instruction is stored in IR. Otherwise, if the cache misses, the fetch unit accesses the main memory and copies a proper cache line to the cache. Cache loading data path is not shown in this figure for simplicity.

The other operation of the fetch unit is to update PC with the correct next address shown as wire PC_NEXT after its computation. There are five candidates for PC_NEXT as follows. The first candidate is obtained by adding four to the PC shown as wire PC_4, which is used when a fetched instruction is 4-byte and predicted as not-taken for

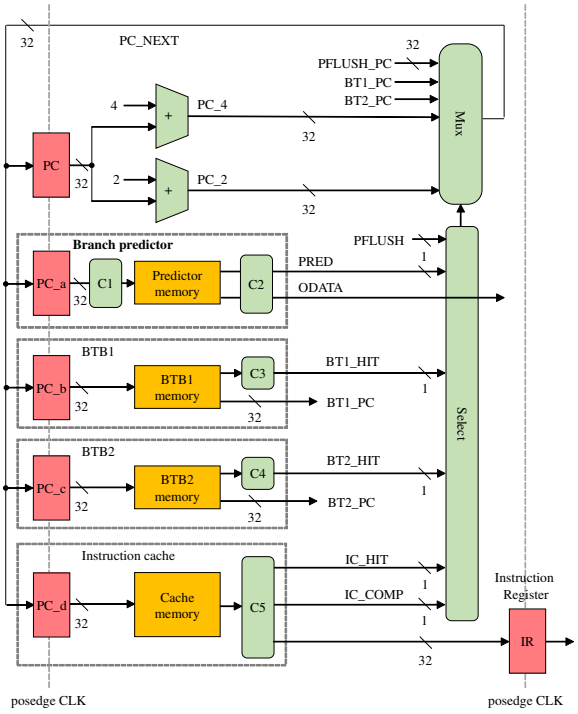


Figure 1. The block diagram of an instruction fetch unit for a RISC-V soft processor supporting the compressed instructions (2-byte instructions). As a branch predictor module, we implement gshare and TAGE based branch predictors.

a branch instruction. The second candidate is obtained by adding two to the PC shown as wire PC_2, which is used when a fetched instruction is 2-byte and predicted as not taken for a branch. The third candidate is branch target address shown as wire BT1_PC from BTB1, which is used when a fetched instruction is a conditional branch and is predicted to be taken. The fourth candidate is the branch target address shown as wire BT2_PC from BTB2, which is used when a fetched instruction is an unconditional branch. The fifth candidate is the correct address shown as wire PFLUSH_PC from the later pipeline stages, which is used when the predicted PC is not correct, or an interrupt occurs. PFLUSH_PC has the highest priority among the five. Note that the wire PC_NEXT is one of the inputs of the branch predictor.

To support the RISC-V compressed instructions of 2-byte width, PC_2 is used. The wire IC_COMP is set when a fetched instruction is a compressed one.

The outputs of the branch predictor are PRED which is the prediction itself and ODATA which is the information used for updating this branch predictor. The branch outcome is determined in the later pipeline stage which is usually EX stage. If the branch prediction is wrong at EX stage, some pipeline registers are flushed and the instruction fetch unit restarts from the correct execution address specified by

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

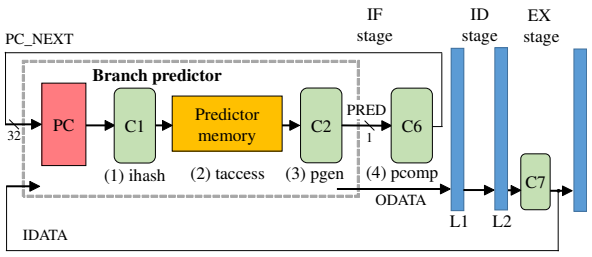


Figure 2. The branch prediction model for FPGA implementation. The signals of IDATA and ODATA are used to map a predictor memory into dual-port RAMs. This is the configuration where the update latency is two (L=2).

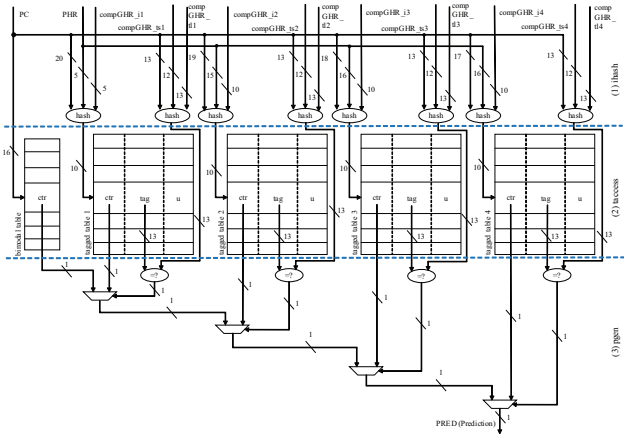


Figure 3. The block diagram of the baseline TAGE branch predictor.

PFLUSH_PC.

The wire BT1_PC and BT2_PC is a branch target address from BTB1 and BTB2, respectively. Each wire of BT1_HIT and BT2_HIT indicates the buffer hit or miss of BTB1 and BTB2, respectively. Each BTB is updated using the actual branch outcome from EX stage. If a fetched instruction is a conditional branch and that branch is taken, then the actual branch target address is stored in BTB1. If a fetched instruction is an unconditional branch, then the branch target address is stored in BTB2.

A multiplexer named Mux selects a proper value from these five candidates, and it is controlled by a unit named Select.

In this figure, as the branch predictor module, we implement gshare and TAGE based predictors. The model of these predictors is described in the next section.

B. Branch prediction model for FPGA implementation

Figure 2 shows our branch prediction model for FPGA implementation. This model consists of some registers, four combinational circuits named C1, C2, C6, and C7, and a predictor memory which consists of some prediction tables. Two input wires of a branch predictor module are PC_NEXT and IDATA, and its output wires are PRED and ODATA. The register PC in a branch predictor module is used to generate the index to access a prediction memory. The registers which are shown as blue rectangles like L1 and L2 are pipeline registers of a soft processor. For example, L1 is the pipeline register between IF stage and ID stage.

The most embedded memories like BRAM (Block RAM) on a Xilinx FPGA are dual-port RAMs. Thus the number of accesses to the RAM in a branch predictor module per clock cycle should be at most two to map a prediction memory into BRAMs efficiently. The naive implementation of a branch prediction requires one read for a prediction, one read and one write for an update in a single clock cycle. Note that the most branch predictors use the increment and decrement of a counter in a prediction table, and the read of a counter and the write a new value to the same counter for an update is necessary. Therefore, the naive implementation requires the three-port memory. To eliminate the read port for updating, we use the signal of IDATA and ODATA. The information of the selected entry, index, and so on which will be used for a predictor update is output as ODATA. Using the branch outcome and ODATA, the combinational circuit C7 generates IDATA that contains the necessary information to update a predictor module.

The signal PRED is the prediction output. In Figure 2, we assume that the updating of a branch predictor is done in EX stage, therefore a branch predictor is updated two cycles later from the prediction in IF stage. As we described earlier, this configuration having the update latency of two cycles is described as L=2. In some processor implementations with a deeper pipeline, the update latency maybe longer, and its effects should be considered. In this model, some pipeline registers of L1 and L2 are used to emulate the configuration of pipelining. When we emulate the L=3 configuration, a pipeline register L3 is inserted between L2 and C7. In general, by inserting n pipeline registers between a branch predictor module and C7, we can emulate the L=n configuration.

As shown in Figure 2, the delay concerning a branch prediction mainly consists of four processes which are (1) index calculation to access tables with hashing in C1, (2) table access in a predictor memory, (3) generate the overall prediction in C2, and (4) computation of PC for the next cycle in C6, which is a set of Mux and Select in Figure 1. These four processes are described as *ihash*, *taccess*, *pgen*, and *pcomp*, respectively. The maximum delay of a naive implementation for an instruction fetch unit concerning a branch prediction is roughly determined by the sum of these four processes.

C. Baseline TAGE branch predictor

Figure 3 shows the block diagram of the baseline TAGE predictor [5], which was proposed by A. Seznec et al. in

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

2006. It has a bimodal table which is indexed by PC shown in the leftmost in the figure. This part is the same as a bimodal branch predictor. It also has four tagged tables that are indexed using different length global histories that form a geometric series. An entry of a tagged table has a saturating 3-bit counter *ctr* whose sign provides the prediction, a tag and 2-bit useful bits *u* to lock entry when the entry provides useful information. By using these four tagged tables, TAGE branch predictor can capture correlation with not only short branch history but also long branch history. In this figure, the rightmost tagged table captures the longest branch history.

At prediction time, these five tables are accessed simultaneously. A bimodal table always provides prediction, and a tagged table provides prediction only when the tag matches. The overall prediction is provided from the matching tagged table with the longest history. When no match occurs, the overall prediction is provided from the bimodal table. It is known that when the provider of the overall prediction is a tagged table and the prediction is made using the weak states of a counter, the accuracy of the prediction is low [7]. In this case, the prediction by strong states of a matching table with the shorter history is often more accurate and used for the overall prediction than one by weak stages with the longest history. This prediction by a table of shorter history is called *alter prediction*. In this work, *alter prediction* is always provided by the bimodal table for the simple implementation.

When a branch outcome is known, the prediction counter of the provider table is updated. The useful bits *u* is updated when the prediction and the alter prediction are different. If a misprediction occurs and the table which provided prediction is not the table using the longest history, then it looks up the entries of tables with a longer history than the provider table for which useful bits *u* is 0. A new tagged entry is allocated into a table with *u*=0, and prediction counter *ctr* is initialized to the weak correct.

In this work, we assign an 18K-bit BRAM to each table, considering the minimum usage of FPGA hardware resources. Therefore, a bimodal table has 8K two-bit counters whose total memory usage is 16K-bit. Each tagged table has 1K entries, and each entry contains a 13-bit tag, 2-bit counter, and 3-bit useful bits, whose total memory usage is 18K-bit. The four tagged tables use the branch histories of 5, 15, 44, and 130 bit, respectively.

The upper part of this block diagram shown in Figure 3 is (1) ihash, and the middle part is (2) taccess, and the lower part is (3) pgen in our branch prediction model described in the previous section.

D. Challenges for TAGE implementation

The TAGE branch predictor has higher prediction accuracy than other conventional predictors like bimodal and gshare. However, there are some challenges to implement a TAGE branch predictor on FPGAs.

Firstly, TAGE uses very long branch history, therefore the area of table memories and computing their indexes and tags takes large area. Note that we use five tables for TAGE, whereas FPGA has two kinds of memories which are BRAM and distributed RAM. However distributed RAM consumes much important resources of LUTs, whereas BRAM doesn't consume LUTs resource. On the other hand, the address of BRAM should be registered and triggered by the edge of the clock, so the implementation of TAGE should be modified to meet this constraint.

Secondly, the naive implementation of a TAGE branch predictor has the sequential processes of (1) ihash, (2) taccess, and (3) pgen. Because these sequence of processes may be the critical path of the instruction fetch unit, the long delay of these processes may lower the operating frequency of soft processors. The implementation of TAGE should reduce the total delay for these processes done in a single clock cycle.

III. Proposal and some TAGE implementations

In this section, we describe a conventional TAGE which uses distributed RAMs which is not practical in terms of hardware resource usage. Then, we describe a TAGE implementation using BRAM. Finally, we propose a branch predictor called pTAGE (pipelined TAGE), which has almost the same prediction accuracy as TAGE and mitigates the drawbacks of the low operating frequency and inefficient hardware resource usage of TAGE.

Note that our proposal is for an efficient implementation of a TAGE predictor for FPGAs, not for the improvement of prediction accuracy of TAGE.

A. Conventional TAGE

The block diagram of the conventional TAGE is shown in Figure 3. Figure 4(a) shows the timing diagram of a conventional TAGE predictor. As described earlier, we divide the process concerning a branch prediction in a clock cycle for a conventional TAGE into four steps including (1) ihash, (2) taccess, (3) pgen, and (4) pcomp. This timing chart shows the behavior executing three predictions (prediction 1, 2, and 3) whose program counters are PC1, PC2, and PC3, respectively. The four steps for a prediction of PC1 are shown in red color. Similarly, the steps of PC2 and PC3 are shown in green and blue colors, respectively. In these timing charts, registers are shown as grey rectangles.

Note that the indexes or addresses to access the prediction tables are calculated in (1) ihash and obtained data is used in (3) pgen, in other words, the adjacent modules of (2) taccess are not registers. Therefore, it does not meet the constraint that the address of BRAM should be registered and triggered by the edge of the clock. So, in the implementation of this timing, not BRAMS but distributed RAMs are allocated for five prediction tables. This implementation uses many LUTs as distributed RAMs and is not acceptable.

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

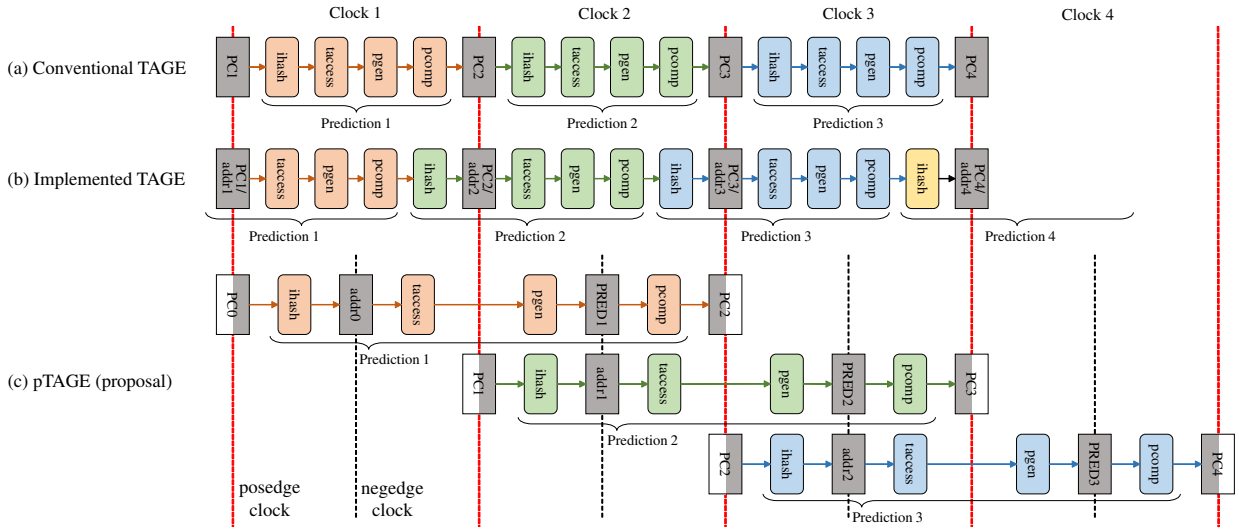


Figure 4. The execution timing of three implementations of a TAGE predictor. (a) is a conventional implementation where prediction memories are not mapped into BRAMs. (b) is the implemented TAGE where prediction memories are mapped into BRAMs. (c) is the proposed implementation named pTAGE using pipelining.

Although it may be obvious, we should note that prediction 1 uses PC1 as input and generates PC2 for the next instruction in the first clock cycle. Then, prediction 2 uses PC2 as input and generates PC3 for the next instruction in the second clock cycle, and so on. Later, you will see that prediction 2 uses **PC1** as input and generates PC3 in our proposal.

B. Implemented TAGE using BRAMs

Since it is highly desirable to implement a TAGE branch predictor with BRAMs which need the registered address as input, the process of ihash is shifted one step left so that taccess can be processed using the address updated at the positive edge of the clock signal.

Figure 4(b) shows the timing diagram of this configuration. Here, we focus on prediction 2 and see the same prediction as the conventional TAGE is obtained using this configuration. At the end of the first cycle, PC2 which was computed using pcomp in the first cycle and the addresses of prediction tables (addr2) which was computed using ihash in the first cycle for prediction 2 are stored in the register *PC2/addr2*. Because addr2 is available in registers, taccess whose input is addr2 of prediction 2 can be processed in the second clock cycle. Then, through pgen and pcomp for prediction 2 in the second cycle, PC3 which is the same predicted PC for the next instruction as the baseline is obtained.

As shown in this figure, the adjacent module of taccess is register and meet the constraint that the address of BRAM should be registered and triggered by the edge of the clock. Therefore, the prediction memories are mapped into BRAMs which is more practical.

We use another optimization for hash computation to reduce the circuit size by using a circular shift register as a folded index of branch history, which is introduced by P.Michaud [8].

This design is called *the implemented TAGE* in this work.

C. pTAGE, a proposed implementation

The implemented TAGE described in the previous section can overcome the limitation of hash circuit size and the memory mapping constraint. However, the problem of low operating frequency remains. To solve this problem, we propose a new implementation of the predictor called *pTAGE*.

Figure 4(c) shows the timing chart of pTAGE. The idea of this implementation is taken from the pipelined branch prediction and the earlier memory access from [9] and [10]. **The key idea is to use PC1 rather than PC2 for prediction 2 to generate PC3. As shown in Figure 4, which makes the slack of one clock cycle because PC1 is available one clock cycle earlier than the timing of PC2 is available.** As shown in the timing chart of pTAGE, a prediction completes in 2 clock cycles. We see that prediction 2 uses PC1 as input and generates PC3 as output in pTAGE.

For the simpler implementation, we use both positive and negative edges of the clock signal. The registers having PC are updated at the positive edge (posedge) and the registers having addresses of prediction memory are updated at the negative edge (negedge) of the clock signal. Within the half clock cycle from a posedge to the next negative edge of the clock, pTAGE processes ihash. For example, ihash of prediction 2 uses PC1 as input, and output is used for addr1 at negedge clock. Similarly, pcomp is processed using the

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

half clock cycle from a negedge to a posedge of the clock signal. Within the whole clock cycle from a negedge to the next negedge of the clock, taccess and pgen are processed. For example, taccess of prediction 2 uses addr1 as input and pgen outputs to PRED2 where both registers are updated at a negedge of the clock signal.

The proposal of pTAGE uses the pipelining where the processes of two predictions are done at the same time. Therefore, although each prediction takes two clock cycles, pTAGE outputs a prediction every clock cycle. In Figure 4(c), we see that PC2, PC3, and PC4 are output at the end of clock 2, 3, and 4, respectively. Note that this timing chart satisfies the data dependency among predictions. For example, PC2 which stores the output of prediction 1 is available at the end of clock 2 where PC2 is used as input of prediction 3 at the beginning of clock 3, satisfying the data dependency.

The pTAGE based instruction fetch unit achieves higher operating frequency than TAGE which may fall into the critical path of the fetch unit. This has a direct effect on the performance of the processor. pTAGE using pipelining enhances the performance of a branch prediction.

IV. Evaluation

The performance evaluation is done by trace-driven simulation in Verilog HDL. We use instruction traces obtained by executing some benchmark programs using Spike RISC-V ISA simulator [11]. We verified the correctness of our model during simulation by comparing the sequence of PC and IR values of retired instructions obtained by the implemented fetch unit and the correct sequence from Spike simulator.

We use 4 benchmark programs including 099.go which is an internationally ranked go-playing program, 129.compress which is an in-memory version of the common UNIX utility, 130.li which is Xlisp interpreter, and 147.vortex which is an object-oriented database from SPEC CINT95 [12]. As our target implementation is soft processors for embedded systems, so we selected SPEC CINT95 which is a relatively small benchmark suite among SPEC CPU benchmarks. Although CINT95 has eight programs, we use four programs which can be compiled with a little code modification using a RISC-V cross compiler, gcc version 7.2.0 using -O2 optimization flag. For each benchmark program, the first 70 million instructions are skipped to exclude its initialization phase and the following 20 million instructions are used for evaluations.

The first performance metric is a miss rate of each branch predictor obtained by dividing the number of mispredictions by the number of total predictions. Another performance metric is the fetched valid instructions per cycle (FIPC) which is obtained by dividing the number of fetched instructions by the total number of elapsed clock cycles.

We compare hardware resources and the frequency of branch predictors for FPGA implementation on xc7a100tcsg324-1 Artix-7 FPGA by using Xilinx Vivado 2017.2. We select the highest frequency which satisfies the clock constraint by changing the clock constraint by 0.2ns steps like 10ns, 9.8ns, 9.6ns, and so on.

Table I
The synthesize and implementation results of three branch predictors targeting Artix-7 FPGA.

	F [MHz]	Slice	Registers	LUTs	BRAMs
Gshare	131.6	26	32	25	8
TAGE	98.0	100	248	242	5
pTAGE	138.9	139	434	263	5

For comparison, we configured the size of prediction memory for gshare branch predictor as 16KBytes (8 BRAMs). And both implemented TAGE and pTAGE branch predictor have five BRAMs. The length of branch history is 16bit for gshare branch predictor and 130bit for implemented TAGE and pTAGE. The implemented TAGE may be referred to as TAGE for simplicity.

A. Hardware resources usage and operating frequency

Table I shows the hardware resources usage and the maximum operating frequencies of three branch predictors. The maximum operating frequencies are obtained using an instruction fetch unit shown in Figure 1 with a branch predictor among three. We selected the very small size of an instruction cache and BTBs to ensure that the path through a branch predictor becomes the critical path. The number of slice, registers, LUTs, and BRAMs have shown here are only for a branch predictor including the updating circuit which is drawn as PC, C1, prediction memory, C2, C6, and C7 in Figure 2.

Firstly, we see the hardware resources usage of predictors. As mentioned before, TAGE and pTAGE consume 5 BRAMs while gshare consumes 8 BRAMs. Because BRAMs are valuable resource of an FPGA, this property of pTAGE is preferable for soft processors. On the contrary, pTAGE consumes more slices, registers, and LUTs compared to TAGE and gshare. Because the organization of gshare is simple, it uses only 26 slices, 32 registers, and 25 LUTs. As shown in Figure 3, TAGE and pTAGE use many hash functions , multiplexers, and comparators. Therefore, TAGE uses 242 LUTs for logic and pTAGE uses 263 LUTs, which are similar to the TAGE usage because both TAGE and pTAGE use almost the same combinational circuits. In terms of register usage, TAGE uses many registers than gshare because of its complex structures having many history registers and path registers. In addition, pTAGE uses more registers for pipeline registers and it consumes 434 registers. Although pTAGE uses many slices, registers, and LUTs, its hardware usage is less than 1 percent of all logic resources for the target FPGA.

Figure 5 and 6 show the place and route results of an instruction fetch unit using gshare and pTAGE, respectively.

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

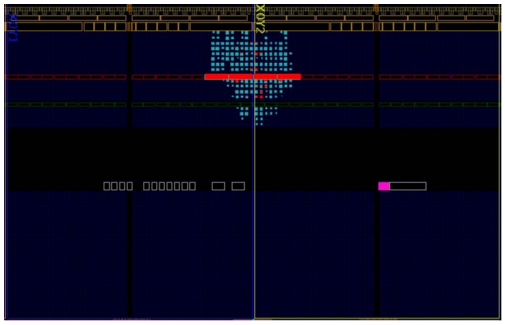


Figure 5. The place and route result of an instruction fetch unit using gshare. Although the target FPGA of xc7a100tcsg324-1 has eight clock regions, we show the two clock regions where the unit was placed.



Figure 6. The place and route result of an instruction fetch unit using pTAGE. Although the target FPGA of xc7a100tcsg324-1 has eight clock regions, we show the two clock regions where the unit was placed.

The hardware resources for a branch predictor are shown in red color. Because the accuracy of a branch predictor is important for a high-performance instruction fetch unit, this hardware increase shown in these figures and Table I will be acceptable.

Then, we see the maximum operating frequencies of an instruction fetch unit using one of three branch predictors. The fetch unit using pTAGE attains 138.9MHz operating frequency which is 41.7% higher than TAGE predictor running at 98.0MHz, and pTAGE achieves better operating frequency of gshare running at 131.6MHz. By using the proposed pipelining scheme, this high operating frequency of pTAGE is obtained.

B. Miss rates of branch predictors

Figure 7 shows the miss rates of three branch predictors varying the update latency L during the instruction fetch and the execution stages from 2 to 22 cycles.

For L=2 configuration which assumes a standard five-stage pipelining, the prediction miss rates of gshare, TAGE, and pTAGE predictors are 8.81%, 7.68%, and 7.68%, respectively. As we expected, the prediction accuracies of TAGE and pTAGE are the same and better than gshare in this configuration.

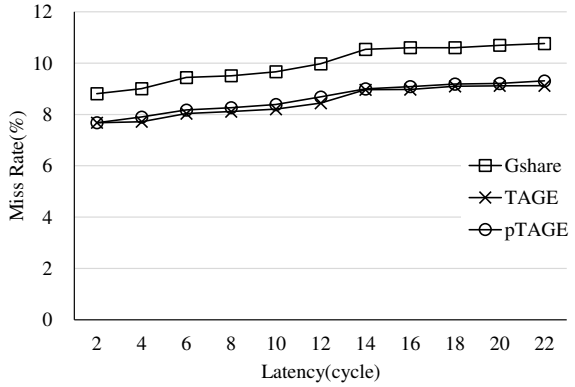


Figure 7. The miss rates of three branch predictors varying the update latency L during the instruction fetch and the execution stages from 2 to 22 cycles. The proposed implementation is described as pTAGE.

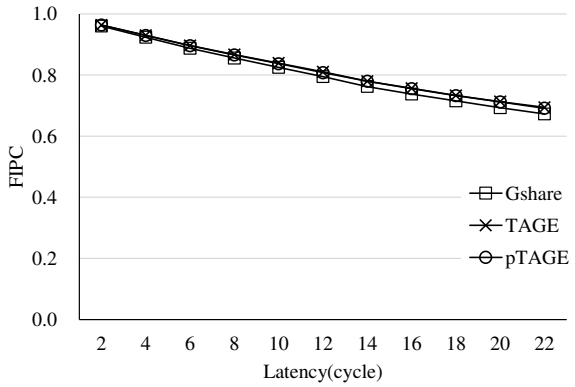


Figure 8. The fetched instructions per cycle (FIPC) of an instruction fetch unit varying the update latency L during the instruction fetch and the execution stages.

As we can see from the figure, the miss rates monotonously increase by increasing L. For L=22 configuration, the miss rate of gshare is 10.77%, which is 1.22 times higher than the gshare of L=2. The miss rate of L=22 TAGE is 9.12%, which is 1.18 times higher than TAGE of L=2. The miss rate of L=22 pTAGE is 9.31%, which is 1.21 times higher than pTAGE of L=2.

The growth ratio of prediction miss rate increasing L for pTAGE is slightly higher than TAGE. However, both TAGE and pTAGE achieve almost the same accuracy, which is much higher than gshare.

C. Fetched valid instructions per cycle (FIPC)

In this section, we show the preliminary evaluation results of the performance of an instruction fetch unit. The evaluation metric is the fetched valid instructions per cycle (FIPC) which is obtained by dividing the number of fetched instructions by the total number of elapsed clock cycles. The

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.

ideal value of FIPC should is 1, where one valid instruction is fetched every clock cycle.

The organization used for evaluation is shown in Figure 1 where an instruction cache is replaced by the ideal cache which never misses to eliminate the overhead caused by the memory accesses. This evaluation includes the effect of BTB misses. The entries of two BTBs are 1,024 and the direct-mapped scheme is used.

Figure 8 shows the evaluation results of FIPC of an instruction fetch unit using gshare, TAGE, and pTAGE, respectively. From this figure, we see that FIPCs decline significantly with increasing L. We also see that FIPCs of both TAGE and pTAGE are almost the same and better than gshare. Especially in large L configuration, the difference between pTAGE and gshare becomes obvious. Although FIPC of three predictors achieves high around 0.93 in L=2 configuration, we should care that pTAGE uses less BRAMs achieving better fetch performance than gshare.

The evaluations of instruction fetch units with pTAGE in a more realistic environment are one of our future works.

V. Conclusion

We proposed the implementation of TAGE branch predictor that keeps the accuracy and performance at a given operating frequency. The pTAGE implementation computes branch prediction in 2 clock cycles, and its processes are pipelined so pTAGE branch predictor provides a prediction per cycle.

We built an environment for evaluation of accuracy, operating frequency, and hardware resources with varying the update latency. Through the evaluation, we found that the prediction accuracy degrades with increasing the update latencies. We showed that pTAGE predictor achieved the same accuracy as TAGE branch predictor, and 41.7% higher operating frequency than TAGE predictor. The proposed pTAGE is efficiently utilizing the FPGA resource while keeping accuracy, higher operating frequency, and lesser usage of BRAMs.

Acknowledgements

This work is supported by VLSI Design and Education Center (VDEC), the University of Tokyo with the collaboration with Synopsys Corporation. This work is supported by JSPS KAKENHI Grant Number JP16H02794.

References

[1] *MicroBlaze Processor Reference Guide*, v2018.2 ed., Xilinx, June 2018.

[2] *Nios II Processor Reference Guide*, Intel, April 2018.

[3] "RISC-V Foundation — Instruction Set Architecture (ISA)," https://riscv.org/.

[4] S. McFarling, "Combining branch predictors," Technical Report TN-36, Digital Western Research Laboratory, Tech. Rep., 1993.

[5] A. Seznec and P. Michaud, "A case for (partially) TAgged GEometric history length branch prediction," *The Journal of Instruction-Level Parallelism*, vol. 8, February 2006. [Online]. Available: http://www.jilp.org/vol8

[6] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.

[7] A. Seznec, "A new case for the tage branch predictor," in *2011 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Dec 2011, pp. 117–127.

[8] P. Michaud, "A PPM-like, Tag-based Predictor," *The Journal of Instruction-Level Parallelism*, vol. 7, April 2005. [Online]. Available: http://www.jilp.org/vol7

[9] D. A. Jimenez, "Reconsidering complex branch predictors," in *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings.*, Feb 2003, pp. 43–52.

[10] A. Seznec and A. Fraboulet, "Effective ahead pipelining of instruction block address generation," in *30th Annual International Symposium on Computer Architecture, 2003. Proceedings.*, June 2003, pp. 241–252.

[11] *Spike, a RISC-V ISA Simulator*. [Online]. Available: https://github.com/riscv/riscv-isa-sim

[12] *SPEC CINT95 Benchmarks*, Standard Performance Evaluation Corporation. [Online]. Available: https://www.spec.org/cpu95/CINT95/

Authorized licensed use limited to: The University of Toronto. Downloaded on October 29,2026 at 14:20:52 UTC from IEEE Xplore. Restrictions apply.